

IN1030

Systemer, krav og konsekvenser

Våren 2026

Obligatorisk oppgave 5

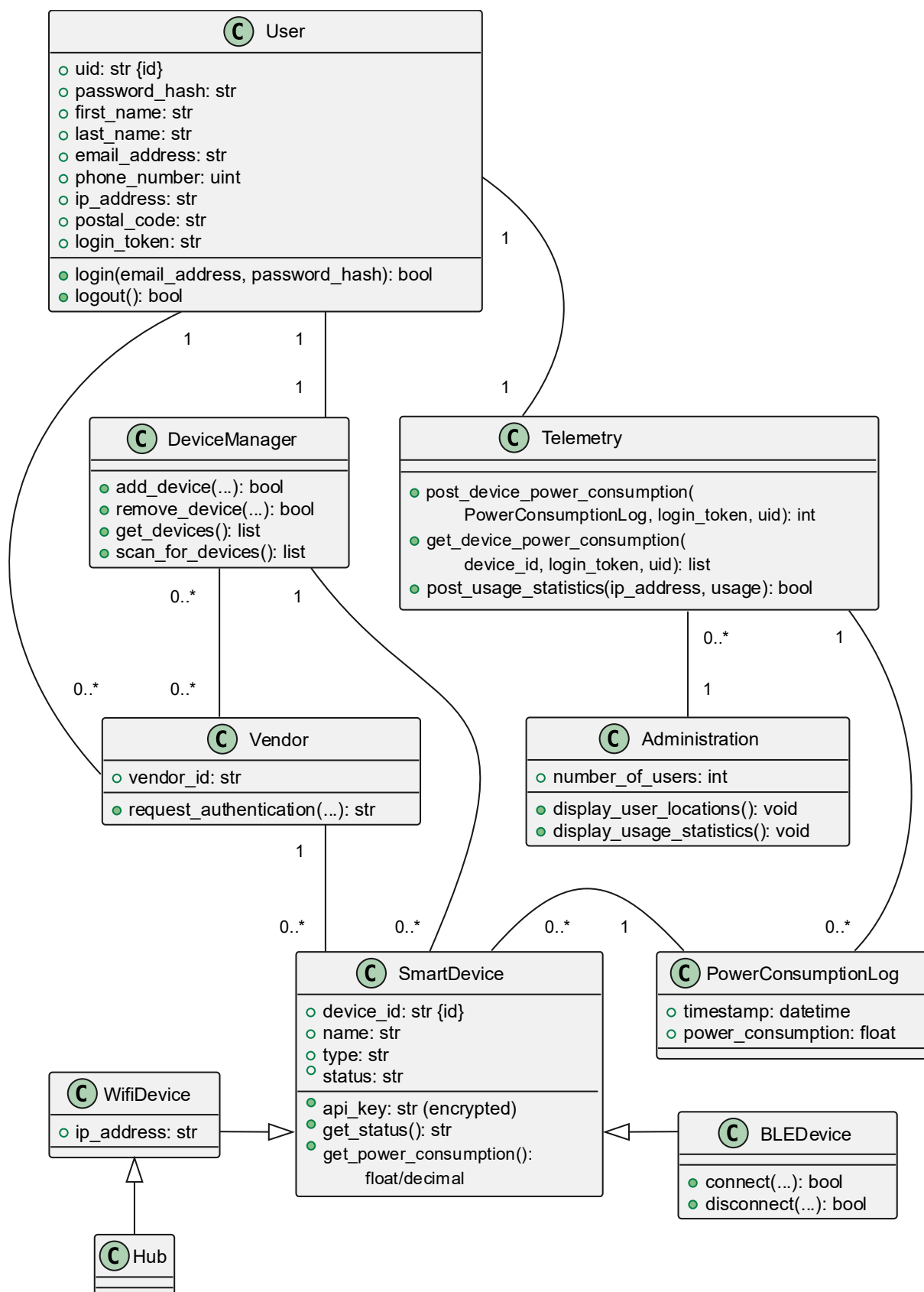
Modellering og krav

Gruppearbeid

Gruppemedlemmer:

Bruker	Personnavn
justassi	Justas Simutis
daniv	Delfine Vågenes
ninmar	Nina Marthinsen
sondrso	Sondre Solheim

Oppgave 1 – Domenemodell (daniv)



Når vi lager et domenediagram er det viktig at programmet oppfyller de kravene som vi har blitt gitt fra Oblig 4. Her var de:

1. Brukerregistrering og innlogging:

-	Brukerne kan registrere seg med navn, e-postadresse og telefonnummer.
-	Brukerne kan deretter logge seg på med e-postadresse og passord.

Dette første kravet for brukerregistrering og innlogging, er gitt ved en klasse *User* med attributter som inkluderer fullt navn, e-postadresse og telefonnummer. Vi inkluderer også en login-metode slik at brukerne kan logge på med epostadressen og passordet sitt.

2. Oversikt over enheter:

-	Brukerne kan få en oversikt over alle smarte enheter i hjemmet.
-	Brukerne skal når som helst kunne legge til nye enheter i systemet.
-	Brukerne skal når som helst kunne slette enheter fra systemet.

** Slikt som lys, termostater, sikkerhetskameraer og døråpnere.*

Dette andre kravet for oversikt over enheter benytter vi oss av to klasser til, hvor vi har *DeviceManager*-klassen som tar seg av å legge til, fjerne eller liste opp enheter som du har koblet til. Vi generaliserer enhetene under *SmartDevice*-klassen, hvor underklasser av denne så kan spesialisere seg mer etter hva slags funksjonalitet den har, men hovedsakelig trenger vi bare en enhet vi kan identifisere som vi gjør gjennom denne klassen. *DeviceManager*-klassen holder så styr på så mange vilkårlige *SmartDevice*-instanser som den krever. Denne *DeviceManager*-klassen er så unik for hver bruker.

3. Rapport og statistikk:

- Brukerne får tilgang til rapport over energiforbruket til smartenhetene i hjemmet, slik at de kan se hvor mye energi de bruker på de ulike enhetene, for å vite hvor de kan spare.

Dette tredje kravet om rapport og statistikk gjøres gjennom en klasse `PowerConsumptionLog`, som holder styr på en `SmartDevice`-instans ved å spørre i intervaller om hjerteslag og hvor mye strøm den bruker. Dette lagrer den så med tidsstempel. Vi systematiserer dette under `Telemetry`-klassen som laster opp all denne dataen til en database i skyen, som kan hentes ned som ønsket i programmet.

4. Administrasjonsmodul:

- Ledelsen skal kunne analysere bruksmønstre og lage rapporter om systemets bruk og popularitet, smat få oversikt over antall personer som bruker appen, og hvor i Norge de er bosatt.

Dette fjerde kravet om administrasjonsmodul gjør seg nytte av den gamle `Telemetry`-klassen, som også sender brukerstatistikk til databasen. Vi legger ved postnummer i `User`-klassen slik at vi også kan ha oversikt over hvor de befinner seg. Administrasjonsmodulen blir så en egen klasse `Administration` som kan hente ned disse dataene fra skyen og visualisere de som ønsket.

En litt forsinket antagelse

En antakelse, som ikke hadde blitt gitt i starten av oppgaven, men i oppgave 2, som var nødvendig for domenemodellen, var jo antakelsen om alle enhetene kobles til nett og kan kobles til gjennom API-nøkler utsendt av leverandoren. Dette gjorde vi først ikke betraktning om og illustrerte hvordan `SmartDevice` kan deles opp i to deler, en for Bluetooth og en for WiFi, ettersom en telefon vil ikke støtte mer enn disse (uten hub). På grunn av denne nye antakelsen, inkluderer vi leverandør som `DeviceManager` vil referere til når en prøver å legge til en enhet. Denne API-nøkkelen blir så lagret i `SmartDevice`-instansen.

Oppgave 2 – Sekvensdiagram

Deloppgave A (ninmar)

Use case

Bruker har glemt passord

Aktører:

-	Brukeren selv
-	SmartNest-systemet (frontend, backend, database)

Pre-betingelser:

-	Brukeren har en registrert konto i SmartNest-systemet.
-	Brukeren har tilgang til e-post-adressen knyttet til kontoen.

Post-betingelser:

-	Passordet til brukerkontoen er nå blitt oppdatert.
-	Brukeren kan deretter logge inn med nytt passord.

Hovedflyt:

1	Brukeren klikker på "Glemt passord" på innloggingssiden.
2	Brukeren skriver deretter inn e-postadressen sin.
3	Systemet sender en e-post med lenke for å tilbakestille passord.
4	Brukeren klikker på lenken i e-posten for å tilbakestille.
5	Brukeren skriver inn nytt passord, i samsvar med passordkrav.

6	Brukeren gjentar det nye passordet som en sikkerhet.
7	Systemet sjekker at de to passord-angivelsene samsvarer.
8	Systemet lagrer det nye passordet for fremtidig pålogging.
9	Brukeren får til slutt en bekreftelse og kan logge inn.

Alternativ flyt 1: Hvis passordene ikke samsvarer:

7.1	Brukeren to passord-angivelser samsvarer ikke med hverandre.
7.2	Systemet viser feilmelding, og brukeren må skrive på nytt.

Alternativ flyt 2: Hvis e-postadressen ikke finnes:

2.1	Oppgitt epost-adresse er ikke registrert på noen bruker.
2.2	Systemet viser feilmelding, og brukeren må skrive på nytt.

Deloppgave B (ninmar)

Use case
Bruker legger til ny enhet

Aktører:

-	Brukeren selv
-	SmartNest-systemet (frontend, backend, database)
-	Enhetsleverandør (sekundæraktør)

Pre-betingelser:

-	Brukeren er innlogget i SmartNest-appen.
-	Brukeren har en konto hos enhetsleverandøren.
-	Den aktuelle enheten er tilgjengelig over Internett.

Post-betingelser:

-	Den aktuelle enheten er koblet til brukerens konto.
-	API-nøkkel fra tredjepart er lagret sikkert i databasen.
-	SmartNest kan kommunisere med enheten via leverandøren

Hovedflyt:

1	Brukeren velger "Legg til ny enhet" i appen (frontend).
2	Frontend sender basert på dette en forespørsel til backend.
3	Backend svarer med å presentere valg av enhetstype.
4	Brukeren velger den typen som svarer til den aktuelle enheten.
5	Systemet videresender brukeren til innlogging hos leverandør.
6	Brukeren logger inn med hos den aktuelle leverandøren.

7	Brukeren godkjenner at SmartNest får tilgang til enheten.
8	Leverandøren sender en API-nøkkel til SmartNest's backend.
9	Backend lagrer denne API-nøkkelen sikkert i databasen.
10	Backend registrerer enheten på brukerens konto.
11	Frontend viser til slutt en bekreftelse til brukeren.

Alternativ flyt 1: Feil innlogging hos leverandør:

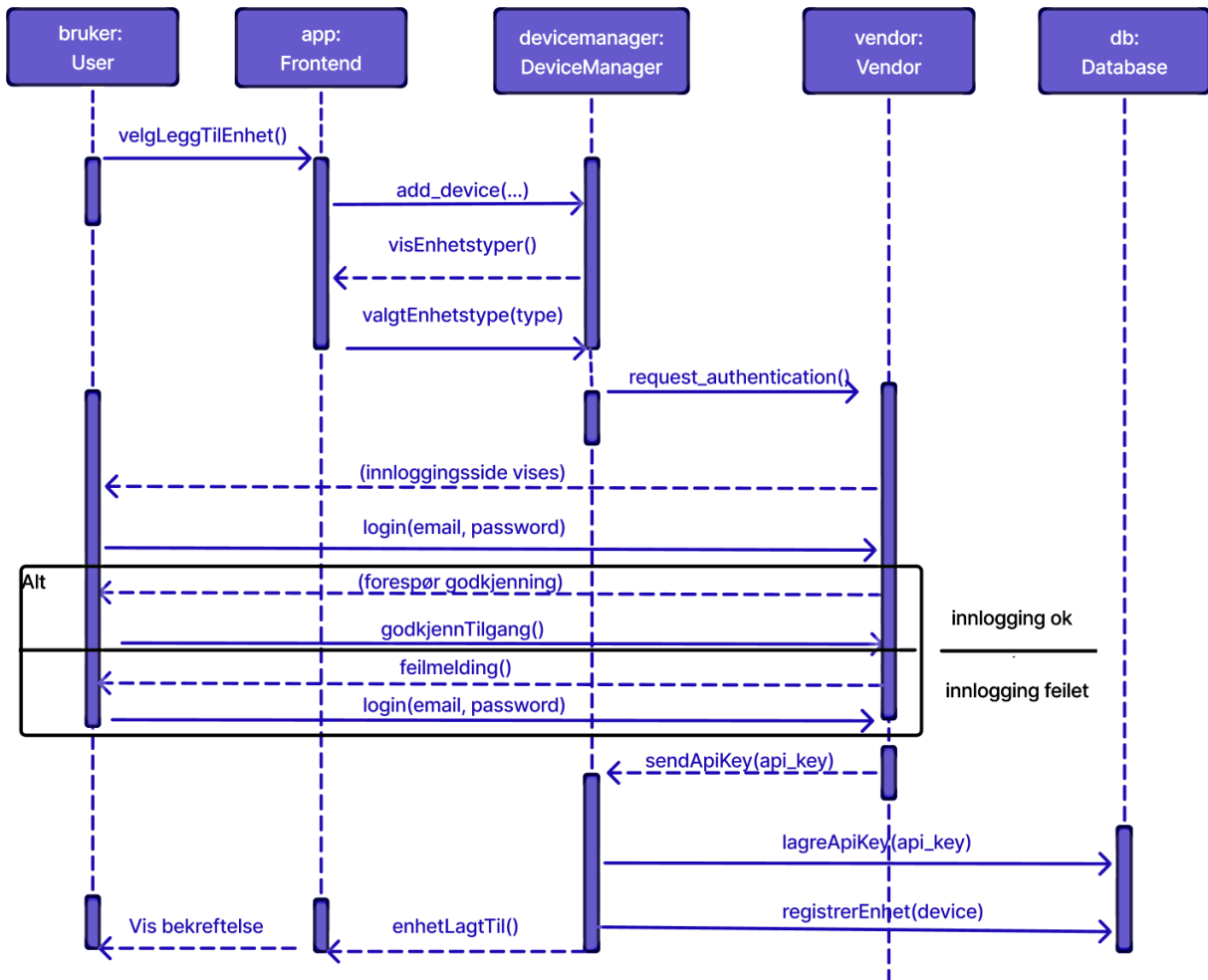
6.1	Brukeren forsøker en feilaktig innlogging hos leverandøren.
6.2	Brukeren får feilmelding og blir så bedt prøve på nytt.

Alternativ flyt 2: Brukeren avslår tilgang for SmartNest:

7.1	Brukeren godkjenner <i>ikke</i> at SmartNest får tilgang til enheten
7.2	Prosessen avbrytes, og enheten blir ikke lagt til.

Deloppgave C (ninmar)

Sekvensdiagram: "Bruker legger til ny enhet"



Oppgave 3 – Aktivitetsdiagram

Deloppgave A (sondrso)

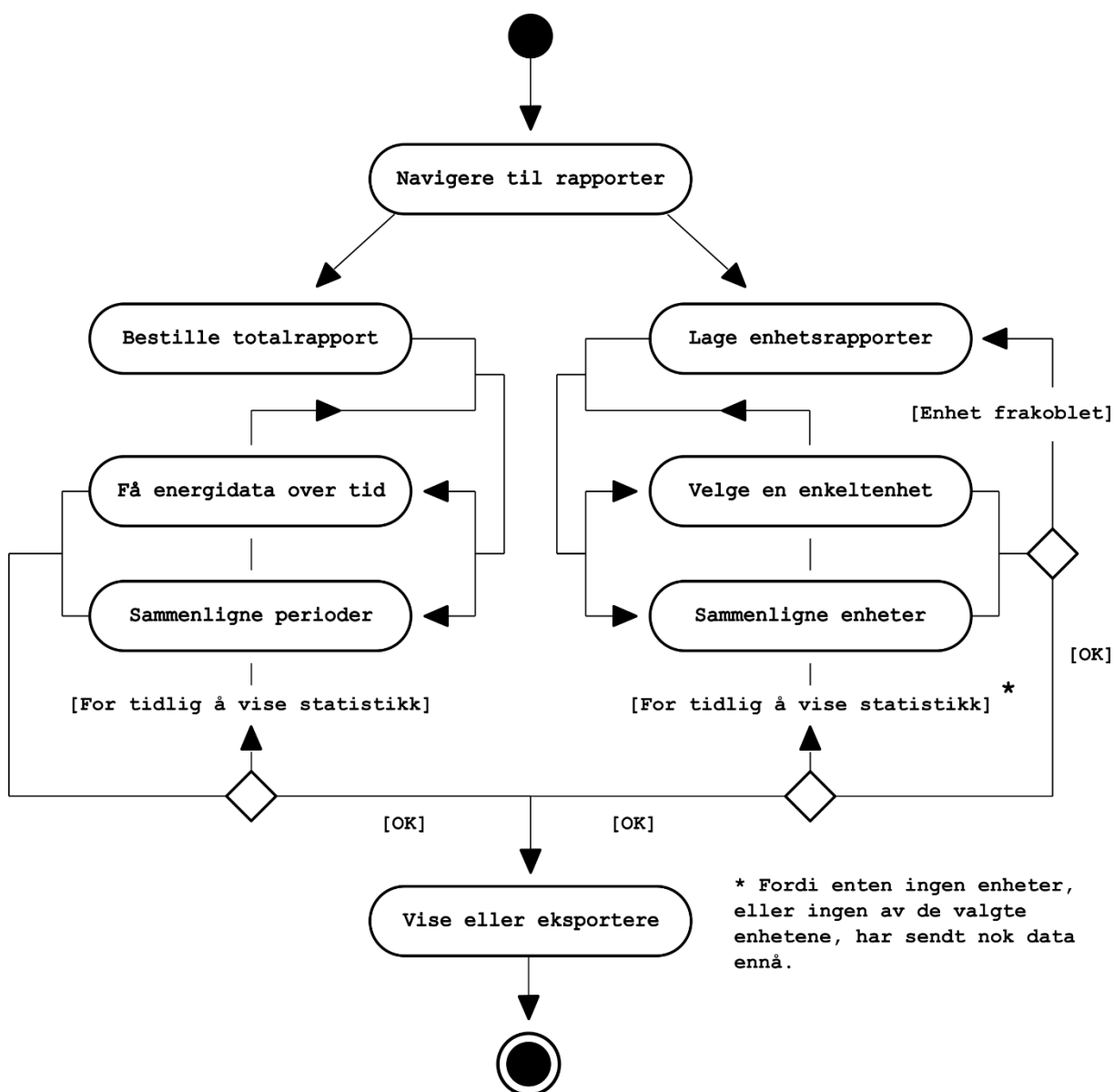
Aktivitetsdiagrammet er en av de tretten diagramtypene definert i UML-standarden, og tjener til å vise hvilke handlinger som kan utføres i systemet, sammen med evt. betingelser, og angivelse av relasjoner, som i denne diagramtypen handler om hvilke andre handlinger som i sin tur muliggjøres. Aktivitetsdiagrammer er konseptuelt sett beslektet med de såkalte "flytskjemaene", fordi de begge beskriver en type kontrollflyt, ved hjelp av bokser og piler, og forgreninger som representerer beslutninger. Forskjellen er at mens flytskjemaet viser kontrollflyten på kodenivå, og dermed mangesmå handlinger og beslutninger som ikke alle hver for seg har noen synlig virkning for brukeren, er det nettopp brukerhandlingene som vises i et aktivitetsdiagram.

Handlingene, her kalt "aktiviteter" tilordnes beskrivende merkelapper innkapslet i avrundede firkanter, og så trekkes det linjer med piler mellom dem. Enkelte steder avhenger de videre mulighetene av en betingelse, ofte symbolisert med en rombe, med vilkårene notert i klammer ved pilene ut fra denne. Det finnes også "barrierer", symbolisert med tykke streker, hvor et sett aktiviteter enten begynner likt og deretter tillates utført parallelt, eller utføres delvis parallelt og alle må være fullført før en etterfølgende aktivitet blir tilgjengelig. Det finnes visst også flere andre, mindre brukte, symboler som vi ikke har lært om. Ofte er det ganske åpenbart hva som må være første og siste aktivitet, men det er uansett god skikk å angi inngang og utgang med henholdsvis en fylt runding og en fylt runding med en sirkel rundt.

Aktivitetsdiagrammer kan tjene flere forskjellige formål, men felles for dem er at man søker å illustrere dynamiske forhold, og hvordan en handling avløser en annen. De er mye brukt til å fremstille arbeids- eller forretningsprosesser, samt bruksprossessen i dataprogrammer. De kan f.eks. illustrere kontrollflyten i et definert brukstilfelle ("use case"), og aktivitetene er da brukerhandlingene. Imidlertid kan de alternativt være programhandlingene, og aktivitetsdiagrammer kan dermed også brukes til å illustrere en énveis databehandlingsprosess. Og på et høyere, mer abstrakt nivå, kan aktivitetsdiagrammer bidra til å kartlegge systemets kontekst, i form av relasjoner som peker ut av det aktuelle systemet og inn mot samarbeidende systemer.

Deloppgave B (sondrso)

Det følgende aktivitetsdiagrammet utdyper i praksis et av de funksjonelle kravene som i forrige innlevering ble definert som brukerhistorier, og senere tegnet inn i diagram over brukstilfeller. I dette diagrammet ble det postulert både en nødvendig og en valgfri underaktivitet, nemlig henholdsvis "hente energidata" og "sammenligne forbruk", og nettopp disse postulatene har vi tatt utgangspunkt i her. Tegningen under inneholder to hovedalternativer som hver gir en valgmulighet i tråd med de to alternativene i brukerhistorien. Deretter følger flere beslutninger, avhengig av om hvorvidt brukerens forespørsel er mulig å oppfylle. Det som ikke fremkommer av aktivitetsdiagrammet er at appen naturligvis vil vise en forklarende feilmelding, som brukeren trykker "OK" på før han eller hun kastes tilbake til et tidligere skjermbilde.



Oppgave 4 – Smidig & DevOps

Deloppgave A (sondrso)

En måte å redusere tiden fra idé til lansering uten nødvendigvis å gå på akkord med systemets kvalitet er, som oppgaven antyder, å anvende en smidig prosessmodell. Dette innebærer at det gjøres minimalt med forhåndsplanlegging og bruks minimalt med tid på dokumentasjon, og at man godtar at veien blir til mens man går. Fokuset vil da ligge på å sette fort i gang med de mest essensielle funksjonene, og lansere disse for brukerne mens man ennå videreutvikler systemet. Det er ikke gitt at den endelige løsningen, i den grad man da i det hele tatt kan snakke om noen endelig løsning, vil utvikles på totalt sett kortere tid, selv om tilhengerne av smidig programmering vil hevde dette, men første versjon blir klar mye forttere.

Smidige prosesser ble utførlig beskrevet i forrige innlevering, og siden det foreliggende systemet trolig ikke vil være så veldig attraktivt før mesteparten av den forutsette funksjonaliteten er på plass, må vi også se på andre momenter. Et stikkord er gjenbruk, et begrep som har blitt veldig sentralt i nesten all systemutvikling. Med dette sikter vi vel å merke ikke først og fremst til hvordan utviklere internt strukturerer kildekoden slik at klasser og funksjoner kan gjenbrukes til lignende oppgaver på forskjellige steder i programmet, selv om god praksis på dette området også kan ha mye å si for tidsbruken. I stedet handler det om å gjenbruke elementer fra tredjeparter, i form av ferdige kodebiblioteker, hele programkomponenter eller nettbaserte programtjenester som ens eget system kan benytte seg av, og dermed skånes for tiden og arbeidet med å programmere alt fra bunnen av.

At denne typen gjenbruk har blitt mer og mer vanlig skyldes både at mengden av løsninger som allerede er blitt implementert av andre, vokser og vokser, og at tilgangen til disse løsningene har blitt forbedret gjennom bla. skytjenester og stadig flere prosjekter med åpen kildekode og frie lisenser. Det krever vanligvis mindre tid å lete seg frem til en gjenbrukbar løsning, og integrere den, enn å programmere noe lignende selv, og finner man komponenter som er blitt tatt i bruk også av mange andre, holder de sannsynligvis høy kvalitet bla. med tanke på sikkerhet og pålitelighet. Imidlertid må man følge med på oppdateringer, og leve med at det kan oppstå brister som skyldes feil i implementeringen hos tredjeparter.

For SmartNests vedkommende er det bla. nærliggende å gjøre en ny vurdering av den avgjørelsen vi tok innledningsvis, hvor vi bestemte oss for å utvikle vårt eget system fra bunnen av i stedet for å basere oss på det allerede eksisterende "Philips Hue". Dette var ut fra en antagelse om at sistnevnte rommer mye mer funksjonalitet enn vi har behov for selv, og at vi var usikre på om løsningen vi kunne kjøpt lisens til ville vært abstrakt nok til at vi lett kunne laget en app som på lik linje støtter forskjellige merkevarer. Begge deler er potensielt et problem, bla. fordi undvending kompleksitet alltid medfører ekstra fare for komplikasjoner.

Deloppgave B (sondrso)

"Continuous integration":

Hver gang det gjøres en endring i kildekoden settes programmet sammen til en kjørbar enhet. Dette innebærer at alle nye eller endrede komponenter kompiles, at man kobler sammen alle programkomponenter, samt alle avhengigheter, i form av f.eks. biblioteker, databaser og konfigurasjonsfiler. Å utføre denne prosessen manuelt er mulig, men tidkrevende, så for å kunne gjøre det ofte, må det automatiseres. Heldigvis finnes det i våre dager en masse støtteprogrammer og integrerte utviklingsmiljøer som kan sørge for dette, og for de fleste programmeringsspråk er det forholdsvis enkelt å kartlegge avhengigheter. Dermed kan prosjektet bygges inkrementelt, dvs. at integrasjonsmekanismen bare bytter ut de delene som er blitt endret siden sist, eller som avhenger av deler som er blitt endret. For å kunne automatisere bygging og integrering, må man vel å merke investere litt tid i å definere og holde oppdatert en såkalt "integration pipeline", som er en oppskrift som forteller programvaren hva som skal gjøres og i hvilken rekkefølge.

"Continuous delivery":

Hver gang det gjøres en endring i kildekoden settes ikke bare programmet sammen til en kjørbar enhet, men installeres også i et realistisk testmiljø, og gjøres til gjenstand for relevante tester både i forkant og etterkant av dette. Det er ikke nødvendigvis snakk om levering til kunde eller sluttbruker, men likevel innebærer det at man til enhver tid besitter en oppdatert utgave av programmet som ikke bare er kjørbar i utviklingsmiljøet, dvs. på datamaskinene til utviklerne, men som også er mulig når som helst å utplassere hos en sluttbruker hvis man skulle ønske det. Et viktig poeng, som forklarer forskjellen mellom kontinuerlig integrering og kontinuerlig levering, er at det kan være store og til dels uforutsette forskjeller mellom utviklingsmiljøet og produksjonsmiljøet, så for å kunne vite tilnærmet sikkert at den aktuelle programversjonen er klar for utplassering, må den på vellykket vis la seg kjøre i et testmiljø med samme platform og innstillinger som produksjonsmiljøet.

"Continuous deployment":

Hver gang det gjøres en endring i kildekoden settes ikke bare programmet sammen til en kjørbar enhet og installeres i et realistisk testmiljø, men utplasseres også med én gang hos sluttbrukeren, som dermed mottar nye oppdateringer like hyppig som utviklerne implementerer endringer i programmet. Dette er gjennomførbart bare hvis sluttbrukernes enheter er kontinuerlig, eller i alle fall tilstrekkelig ofte, tilkoblet en skytjeneste som laster ned slike oppdateringer automatisk, slik vi etter hvert er blitt vant til med operativsystemer. Både kontinuerlig integrering, kontinuerlig levering og kontinuerlig utplassering forutsetter, i stigende rekkefølge, automatisering, som i sin tur avhenger av signaler som trigges når det opprettes en ny versjon i hovedgrenen av programmet under utvikling. Denne teknologien kalles "webhooks", og støttes av bla. Github, et vanlig system for kildekodeforvaltning. Merk at ikke alle bedrifter som utvikler programvare ser seg tjent med å faktisk utplassere alle endringer fortløpende, og at mange kunder og brukere heller ikke nødvendigvis setter pris på det.

Oppgave 5 – Risikohåndtering (justassi)

	Risikobeskrivelse	Sannsynlighet	Konsekvens
1	En nøkkelutvikler blir syk eller slutter midt i prosjektutviklingen.	LAV	HØY
2	Funksjonaliteten viser seg å ta lengre tid å utvikle enn estimert.	HØY	MIDDELS
3	Diverse sikkerhetsbrudd, som f.eks. uautorisert tilgang til enhetene.	MIDDEL	HØY
4	Tredjeparts-API som systemet er avhengig av, slutter å fungere.	LAV	HØY
5	En godt etablert konkurrent lager et lignende system billigere eller gratis.	MIDDELS	HØY
6	Dårlige anmeldelser fordi brukerne ikke er fornøyd med systemet.	MIDDELS	MIDDELS

	Forslag til tiltak	Risikotype	Ansvarlig
1	Ha minst to utviklere som kjenner til samme del av kodebasen.	Prosjekt	Prosjektleder
2	Prioritere de viktigste delene først, og gjøre realistiske estimer.	Prosjekt	Prosjektleder
3	Kryptere enhetskommunikasjon og personopplysningene i databasen.	Produkt	Sikkerhetsutvikler
4	Minimere bruken av tredjeparts-API'er og følge med på oppdateringer.	Produkt	Backend
5	Ikke prise oss for høyt, samt sørge for unik funksjonalitet og design.	Forretning	Produkteier
6	Bla. teste implementasjonen av universell utforming tidlig i løpet.	Produkt	Frontend